Programação Concorrente

Escalonamento de Processos

Professor: **Jeremias Moreira Gomes**
E-mail: jeremias.gomes@idp.edu.br

Escalonamento de Processos

Introdução

- Em computadores multiprogramados, é comum a disputa por CPU
  - Ocorre por dois processos estarem no estado de pronto
- De alguma forma, precisa-se escolher quem será executado em seguida
  - Qual critério utilizar?
- Essa escolha é feita pelo **Escalonador**, que utiliza um **Algoritmo de Escalonamento** para decidir

Introdução

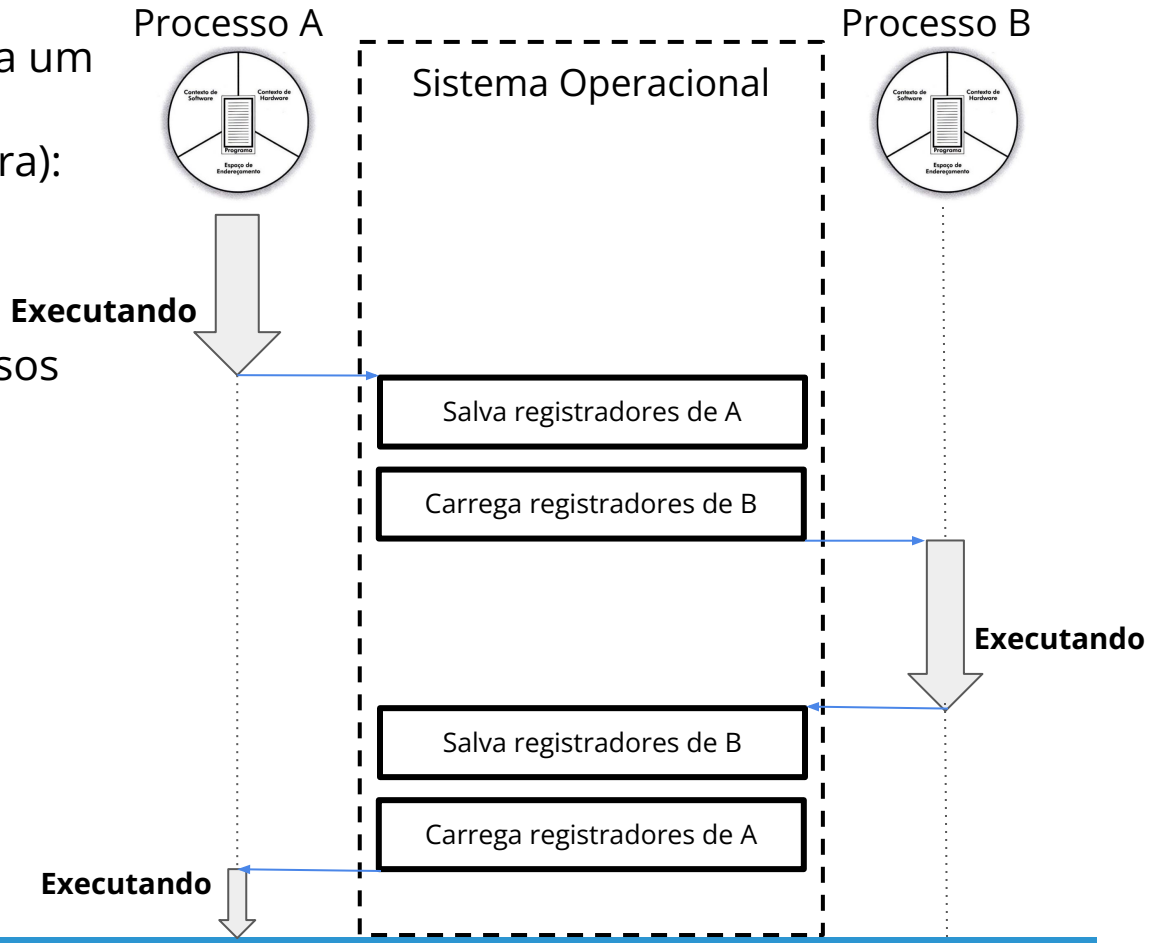
- Como CPU é um recurso escasso, escalonamento é diferencial para o desempenho e também satisfação do usuário
- Aleḿ da escolha do processo certo, é necessário preocupar-se com **uso eficiente da CPU**
    - **Chaveamento de processos é um processo custoso**

Escalonamento de Processos

- **Escalonador**
  - A troca de processos é feita pelo **Escalonador de Processos**
  - Escalonador é o processo que escolhe **qual será o próximo processo a ser executado**
  - Existem diversas técnicas (algoritmos) para realizar o escalonamento de processos
  - Nível mais baixo do Sistema Operacional

Mudança de Contexto

- A mudança de contexto leva a um *overhead* de tempo (tarefa cara):
  - É preciso salvar as informações dos processos movimentados
  - Salvar o conteúdo dos registradores



Escalonamento de Processos - Componentes

- **Despachante** (*Dispatcher*):
  - Armazena e recupera o contexto
  - Atualiza as informações na Tabela de Processos
  - Processo relativamente rápido
- **Escalonador** (*Scheduler*):
  - Escolhe a próxima tarefa a receber o processador
  - Tarefa mais demorada

Escalonamento de Processos - Escalonador

- **Quando o Escalonador é acionado?**

  a. **Quando um novo processo é criado**

     - Quando um novo processo é criado, quem (pai ou filho) fica com o processador?

  b. **Um processo chegou ao fim e um processo pronto deve ser executado**

  c. **Quando um processo é bloqueado (dependência)**

Escalonamento de Processos - Escalonador

- **Quando E/S ocorre, o Escalonador deve (opções):**

  a. Executar o processo que estava esperando esse evento

  b. Continuar executando o processo que já estava sendo executado

  c. Executar um terceiro processo que esteja pronto para ser executado

Escalonador de Processos - Categorias

- **Preemptivo:**

  a. Quando um processo pode perder uso da CPU

     - Provoca uma interrupção forçada de um processo para que outro possa utilizar a CPU

- **Não Preemptivo**:

  a. Permite que o processo sendo executado continue executando

Escalonador de Processos - Categorias

- Condições de para o escalonador **Não Preemptivo**:

  a. O processo terminar a execução

  b. O processo solicitar operação de E/S

  c. Liberação explícita do processador

Algoritmos de Escalonamento

- Cada categoria tem um objetivo diferente para o escalonamento
- São separados em três categorias:

a. **Sistema em Lote** (*Batch*)

   - Vazão (throughput), tempo de retorno e utilização de CPU

b. **Sistemas Interativos**

   - Resposta rápida e proporcionalidade

c. **Sistemas em Tempo Real**

   - Cumprimento de prazos e previsibilidade

Algoritmos de Escalonamento

- Independente da categoria, algoritmos também devem:

  a. **Ser Justos** - Dar a um processo a porção justa de CPU

  b. **Aplicadores da Política** - Manter a política de escalonamento funcionando

  c. **Mantenedores do Equilíbrio** - Garantir que todas as partes do sistema estejam ocupadas

Algoritmos de Escalonamento

1. **Sistema em Lote (*Batch*)**

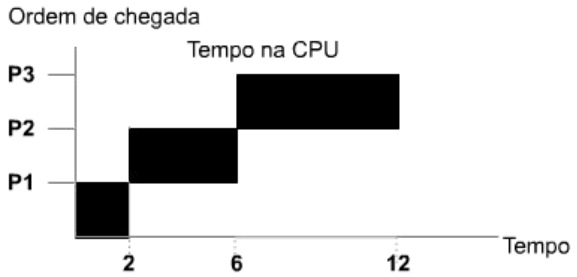
   a. *First-Come First-Served* (*first in, first out*) (FIFO)

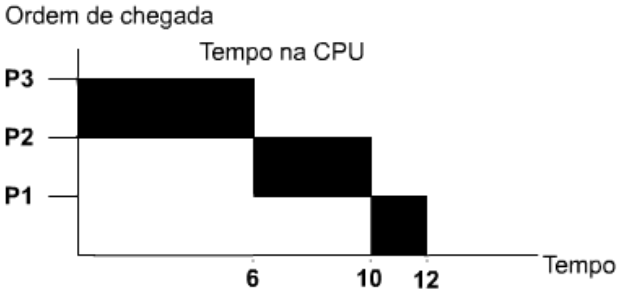
   b. *Shortest Job First* (SJF)

   c. *Shortest Remaining Time Next* (SRTN)

Algoritmos de Escalonamento - FIFO

- Não preemptivo
- Processos são executados na CPU seguindo a ordem de requisição
- Fácil de entender e programar
- Principal Desvantagem:
  a. Ineficiente quando há processos que demoram na CPU





*Ineficiente do ponto de vista de tempo de espera médio

Algoritmos de Escalonamento - SJF

- Não preemptivo
- Deve-se prever o tempo de execução do processo
- Menor processo da lista é executado primeiro
- Possui menor ***turnaround***
    a. Média dos tempos de espera considerando também a execução
- Desvantagens:
    a. Todos os *jobs* precisam ser conhecidos de antemão
    b. Se *jobs* curtos começarem a chegar, os longos podem nunca ser executados

Algoritmos de Escalonamento - SJF

- ***Turnaround***:

8	4	4	4
A	B	C	D

Execução em Ordem (FIFO):

- *Turnaround* de A: 8
- *Turnaround* de B: 12
- *Turnaround* de C: 16
- *Turnaround* de D: 20

Média -> 56 / 4 = 14

4	4	4	8
B	C	D	A

Execução utilizando SJF:

- *Turnaround* de A: 4
- *Turnaround* de B: 8
- *Turnaround* de C: 12
- *Turnaround* de D: 20

Média -> 44 / 4 = 11

*Considerando que todos os processos chegaram na CPU ao mesmo tempo (tempo 0)

Algoritmos de Escalonamento - SRTN

- Preemptivo
- Versão preemptiva do SJF
- Processos com menor tempo de execução são executados primeiro
- Se um processo novo chega e seu tempo de execução é menor do que o do processo corrente na CPU, a CPU suspende o processo corrente e executa o processo que acabou de chegar

Algoritmos de Escalonamento - SRTN

- Desvantagens:
  - Processos que consomem mais tempo podem demorar muito para serem finalizados
    - Muitos processos pequenos passando na frente (inanição)

Algoritmos de Escalonamento

2. Sistemas Interativos

a. Round-Robin

b. Prioridade

c. Múltiplas Filas

d. Shortest Process Next

e. Garantido

f. Lottery

g. Fair-Share

Algoritmos de Escalonamento - *Round-Robin*

- Mais simples e mais utilizado
- Preemptivo
- Cada processo recebe um tempo de execução chamado ***quantum***
  - Tempo compartilhado de mesmo tamanho para processos
- Ao final desse tempo, o processo é suspenso e outro entre em execução
- O processo suspenso vai para o final da fila
- Escalonador mantém uma fila circular de processos prontos

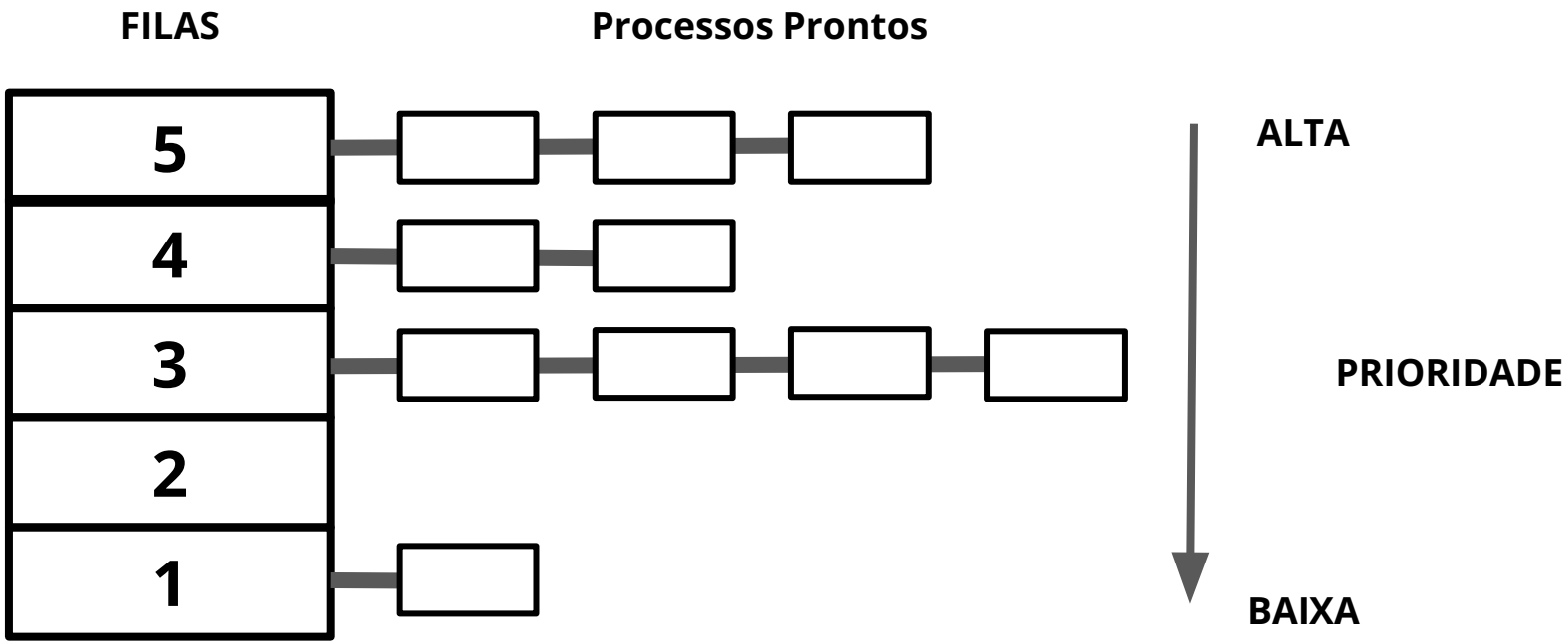
Algoritmos de Escalonamento - *Round-Robin*

- Desvantagens:
  - Qual o tempo ideal de chaveamento entre processos?
    - Lembrar que há o tempo do *dispatcher* e o tempo do escalonador
  - *Quantum*:
    - Se for muito pequeno, ocorrerão muitas trocas de processos, diminuindo a eficiência da CPU
    - Se for muito longo, o tempo de resposta pode ser comprometido

Algoritmos de Escalonamento - Prioridades

- Preemptivo
- Cada processo possui uma prioridade
- Processos prontos com maior prioridade são executados primeiro
- Prioridades são atribuídas dinamicamente
    - Pelo sistema ou estatisticamente
- Round-Robin pressupõe igual prioridade para todos os processos

Algoritmos de Escalonamento - Prioridades



Algoritmos de Escalonamento - Prioridades

- Enquanto houver processos na classe maior, estes são executados
- Classe de menor prioridade só são atendidas quando as de classe superior não precisam mais de processamento
- Classes menos prioritárias podem nunca ser atendidas (inanição):
  - Ajuste de prioridades (de alguma forma)

Algoritmos de Escalonamento - Múltiplas Filas

- Cada vez que um processo é suspenso, ele recebe mais tempo (dobra o *quantum*) na próxima execução
- Ao sair da execução, ele muda de classe (prioridade mais baixa) e é suspenso
- Reduz o número de trocas de processo
- Processos mais longos vão recebendo mais tempo de processador
- Preemptivo

Algoritmos de Escalonamento - Múltiplas Filas

- Exemplo de um processo com necessidade de 100 *quanta*
  - Recebe 01 *quantum* na fila de prioridade mais alta (10, por exemplo)
  - Executa por 01 *quantum* e é suspenso (não terminou)
  - É alocado na fila de prioridade de um nível abaixo (09) com 02 *quanta*
  - Executa por 02 *quanta* e é suspenso (não terminou)
  - É alocado na fila de prioridade de um nível abaixo (08) com 04 *quanta*
  - ... (continua até terminar a execução) ...

Algoritmos de Escalonamento - Múltiplas Filas

- Quanto mais próximo de ser finalizado, menos frequente é o processo na CPU
- É uma técnica eficiente pois processos pequenos tem sua vez
- Também sofre a possibilidade de inanição

Algoritmos de Escalonamento - SPN

- Mesma ideia do SJF
- Processos interativos: não se conhece o tempo necessário para execução
- Utiliza estimativa de tempo baseada em execuções antigas de tarefas similares
- Baseado em verificação de comportamento

Algoritmos de Escalonamento - Garantido

- Garantias são dadas aos processos dos usuários
- n usuários, então $\frac{1}{n}$ do tempo de CPU é dado para cada usuário

Algoritmos de Escalonamento - Loteria

- Cada processo recebe “bilhetes” que lhe dão direito a recursos do sistema
- Quando um escalonamento deve ser feito, escolhe-se aleatoriamente um bilhete
- Processos mais importante podem receber mais bilhetes

Algoritmos de Escalonamento - Loteria

- Precisa garantir que todos terão vez no recurso.
- Uma das soluções para distribuição de recursos é manter duas filas:
  - Bilhetes já sorteados
  - Bilhetes não sorteados
- Quando esgotam-se não sorteados, o esquema reinicia

Algoritmos de Escalonamento - *Fair-Share*

- O dono do processo é levado em conta
- Um dono recebe a sua fatia de processo prometida, independente do número de processos
  - Exemplo com 50/50 entre 2 usuários:
    - Usuário 1 -> A, B, C, D
    - Usuário 2 ->E
    - Execução: A, E, B, E, C, E, D, E, A, E, B, E, ...
  - Mesmo exemplo, mas com ⅔ para o usuário 1:
    - Execução: A, B, E, C, D, E, A, B, E, ...

Algoritmos de Escalonamento

3. **Sistemas de Tempo Real**
   - Tempo é um fator crítico.
   - Computador deve reagir apropriadamente dentro de um intervalo fixo de tempo
   - Sistemas Críticos:
     - Piloto automático de aeronaves
     - Monitoramento de pacientes em hospitais
     - Controle e automação em fábricas

Algoritmos de Escalonamento

3. Tipos de Sistemas de Tempo Real
    - ***Hard Real Time***: atrasos não são tolerados
    - ***Soft Real Time***: atrasos ocasionais são tolerados

Algoritmos de Escalonamento

3. Sistemas de Tempo Real
   - Eventos causam a execução de processos
   - Quando um evento externo é detectado, cabe ao escalonador escalonar os processos de modo que **todos os prazos sejam cumpridos**
   - Eventos são classificados como:
     - Periódicos
     - Aperiódicos

Algoritmos de Escalonamento

3. Sistemas de Tempo Real
   - Algoritmos são divididos entre:
     - **Estáticos**: decisões de escalonamento antes do sistema começar a executar.
     - **Dinâmicos**: decisões de escalonamento em tempo de execução.

O que os sistemas utilizam hoje?

- **Linux**
  - O(n) scheduler e O(1) scheduler (2001 a 2007)
    - Baseados em epoch e tempo
  - CFS (Complete Fair Schedule) (2007 a 2023)
    - Árvores Red-black por processador
  - EEVDF (Earliest eligible virtual deadline first scheduling) (2023)
    - Prioridade baseado em tempo virtual

Conclusão